Team and Tools

Dirk Riehle, FAU Erlangen

AMOS B01

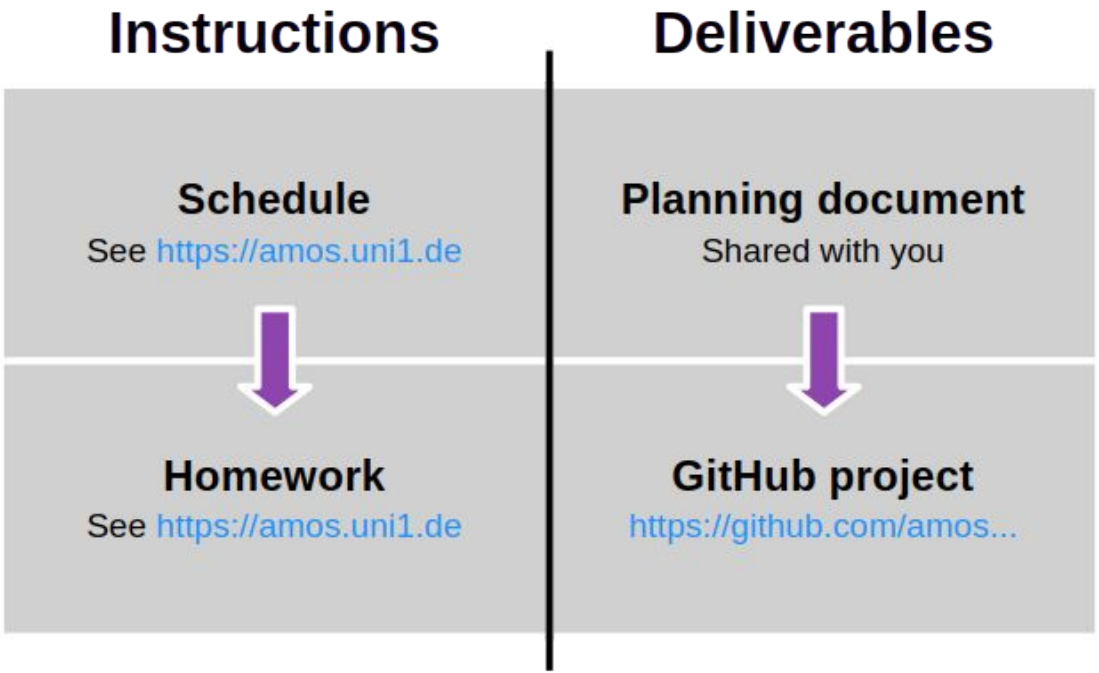
Licensed under CC BY 4.0 International

Agenda

1. Team formation
   a. Student roles
   b. Team contract
   c. Team logo and T-shirt
2. Project tools
   a. Planning document
   b. Feature board
   c. Code repository
3. Team coordination
   a. Imp-squared backlog
   b. Stand-up emails
   c. Happiness index

Instructions and Deliverables

DR



1. Student Roles

The AMOS Scrum Team

DR



https://uni1.de/amos

Scrum Roles

Product owner

- Holds overall responsibility that the software is valuable (to customers)

Software developer

- Holds overall responsibility for design and implementation of the software

Scrum Master

- Holds overall responsibility for continuous process improvement

Additional Roles

Release manager

- Holds overall responsibility that a demo is ready for review

There is no Project Manager

Scrum teams are self-organizing

First Industry Partner Meeting

You should have a first meeting with your industry partner as soon as possible

- Team members in general participate to learn about project
- Product owners participate to gather requirements
- Software developers participate to ask technical questions

After the first team meeting you should still try to regularly meet the industry partner

- As many team members should participate as is sensible and possible

2. Team Contract

Team Contract

A team contract is

- A contract between team members on how to conduct the project

Its main components are

- Goals and rewards
- Norms and sanctions

Goals and Rewards

Goals are what the team hopes to achieve; there are several categories:

- Learning objectives, for example,
  - To ensure that all team members understand the course material
- Interpersonal relationship objectives, for example,
  - To foster an atmosphere of mutual respect and learning
- Functional objectives, for example,
  - To have efficient team meetings

Rewards are how you celebrate intermediate or final achievements

- Let’s have cake!

Norms and Sanctions

Norms are rules for expected behavior; there are several categories:

- Meeting norms, for example,
  - Is being late to a team meeting acceptable?
- Working norms, for example,
  - How will we make decisions?
- Coordination norms, for example,
  - Who keeps meetings on track?
- Communication norms, for example,
  - How to communicate outside of team meetings?

Sanctions are what to do if norms are violated

- Sing a song to the team or do ten push-ups

One-Time Deliverable: Team Contract

Please discuss and agree on a team contract (in planning document)

Try to finish this during the first team meeting

3. Team Logo and T-shirt

Team Logo

A team logo is just that, a logo that

- Represents your team and project

The logo will be used in different places

- On your team T-shirt
- In the GitHub documentation
- In your final demo and report
- Wherever you see fit



Team Logo Design

Get started with collaboratively designing the team logo during the team meeting

Continue on your self-chosen communication channels

Final execution (graphics) may be delegated to one person

Team T-shirt

Create a team T-shirt design using your logo

- You can add your logo and one text

Submit your team T-shirt preferences

- Color, size, form



One-Time Deliverable: Team Logo / Team T-Shirt

Please create the team logo and T-shirt design and submit your preferences

4. Planning Document

The Planning Document

The planning document serves to

- Document basic project information
- Collect all materials that don’t go easily into the repository
- Coordinate work on this less frequent information

See also the Flowers example planning document

Table of Contents

#	Artifact name (tab in spreadsheet)	Artifact purpose
1	Project data	Provides basic project data
2	Project team	Shows project team
3	Role assignments	Tracks role assignment
4	Team contract	Shows team contract
5	Product goal	Provides product vision and project mission
6	Product glossary	Provides domain terminology of project
7	Sprint goals	Lists goals of respective sprints
8	Velocity tracking	Tracks realized story points per sprint
9	Mid-project release tracking	Tracks mid-project release sprints
10	Final project release tracking	Plans and tracks final project release sprints
11	Definition of done	Provides decision criteria for “done”
12	Documentation	Provide links to documentation of product
13	Bill of materials	Lists all third-party components

.de

1. Project Data

Please fill in the data as needed and as you see fit

Please do not protect the online Zoom team meeting

Please do not change the Zoom link during the semester

2. Project Team

Please provide your name and GitHub id

- Almost all work takes place in the project's GitHub repository
- Please create and provide your GitHub id as soon as possible

Please **use only one GitHub id** during the semester

3. Role Assignments

Please fill in the roles people play in a given week (sprint)

Please finish this during the first team meeting

Project Deliverables

Please provide all deliverables (homework) in the

- **Deliverables** folder of the project's GitHub code repository

Details can be found in the homework instructions

Regular Deliverable: Planning Document

Please initialize your planning document and keep it up-to-date

Terms (The GitHub Terminology Mess)

Agile / Scrum	AMOS	GitHub	Other
Project	**Project**	Repository	–
Backlogs [1]	**Backlogs**	Columns in project [2]	–
Kanban board [3]	**Feature board**	Project	–
Backlog item [4]	**Backlog item**	Item, also issue [5]	Work item, ticket
–	**Code repository**	Code	–

[1] Backlogs can be of different types: Product backlog, sprint backlog
[2] The columns of a GitHub project represent the different backlogs + states of work
[3] Scrum proper does not know kanban boards, but agile in general acknowledges them
[4] Backlog items can be of different types: Feature [6], refactoring, bug fix
[5] Terms vary throughout the GitHub user interface
[6] Features should be presented using the user story format

5. Feature Board

Feature Board (GitHub Project)

A feature board is a slotting system used to

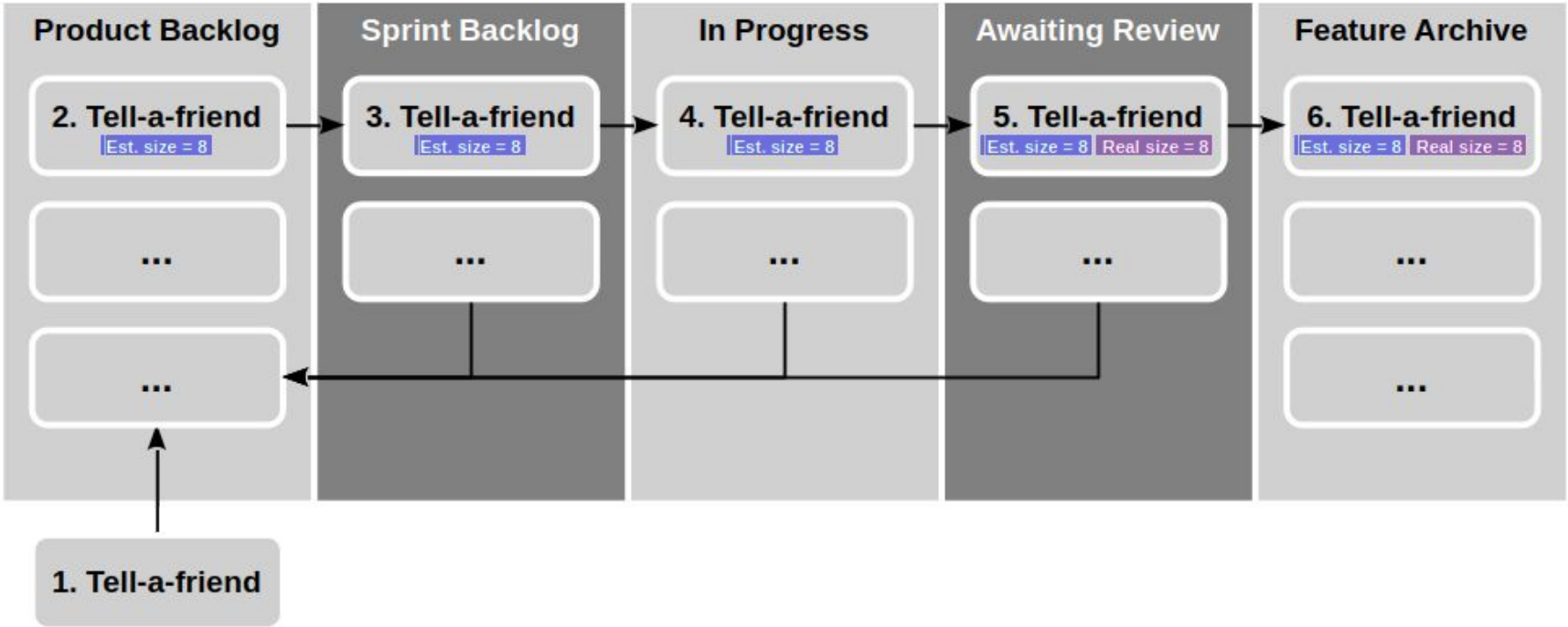
- Manage the processing state of backlog items

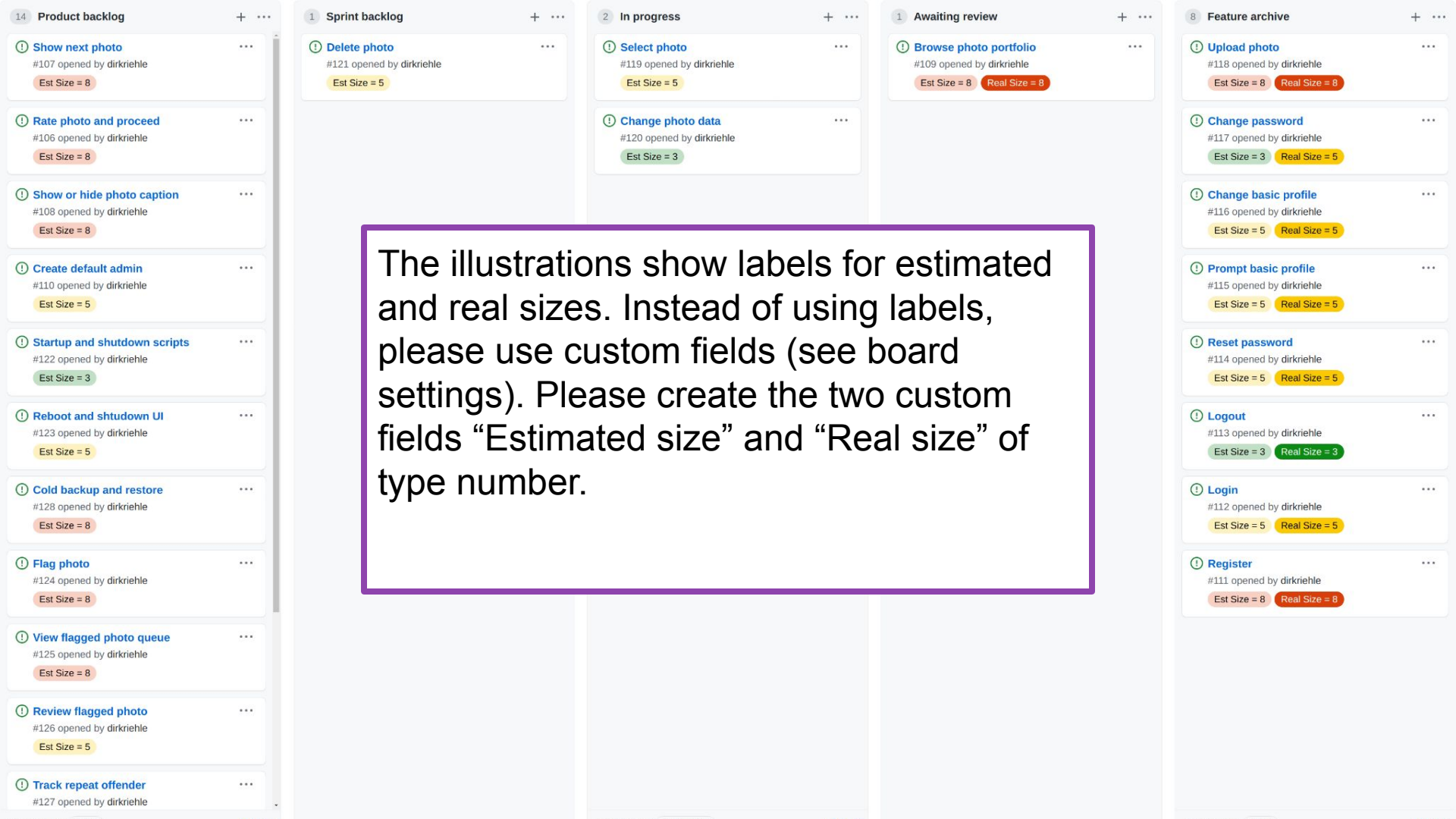
Types of slots (work process states, represented by columns)

- Product backlog (needs doing)
- Sprint backlog (ready to be done)
- In progress (being worked on)
- Awaiting review (needs sign-off)
- Feature archive (finished and archived)

Please do not change the feature board’s setup

Example Feature Board and Backlog Items





The illustrations show labels for estimated and real sizes. Instead of using labels, please use custom fields (see board settings). Please create the two custom fields “Estimated size” and “Real size” of type number.

Backlog Items (Also: Issue, Work Item, Ticket)

A backlog item is an

- Ideally semantically closed **work that needs doing**

Types of backlog items

- **Features** (functional and non-functional user requirements)
- **Refactorings** (behavior-preserving code improvements)
- **Bug fixes** (fixes to malfunctioning code)

We use GitHub issues to represent backlog items

- Either by directly entering them into the GitHub project (the feature board)
- Or by entering them as an issue, also assigning them to the feature board

Example Feature Using User Story Format

Open **Tell a friend** #129

dirkriehle opened this issue 2 days ago · 0 comments



dirkriehle commented 2 days ago • edited

Owner

User story

1. As a guest
2. I can email a friend about a photo
3. So that I can share my passion

Acceptance criteria

- After hitting OK, an email is sent
- After hitting Cancel, no email is sent
- The next page is the main page

Definition of done

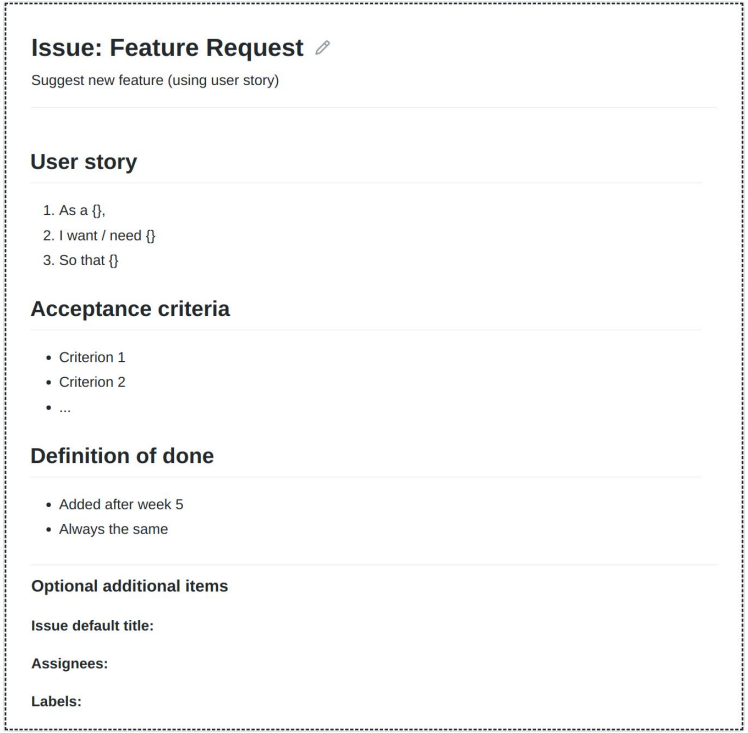
To be defined week 5 onwards.

Using Issue Templates for Features

Please create a template for feature requests

- The template should contain fields for
  - Short name (already preset as title field)
  - Short description (using user story)
  - Acceptance criteria (to test for fulfillment)
  - Definition of done (from sprint 5 on)
- Create custom fields for features sizes
  - **Estimated size = X** for estimated sizes
    - Set during sprint planning
  - **Real size = Y** for actual size
    - Set during sprint review

Feel free to add other templates



Issue: Feature Request

Suggest new feature (using user story)

User story

1. As a {},
2. I want / need {}
3. So that {}

Acceptance criteria

- Criterion 1
- Criterion 2
- ...

Definition of done

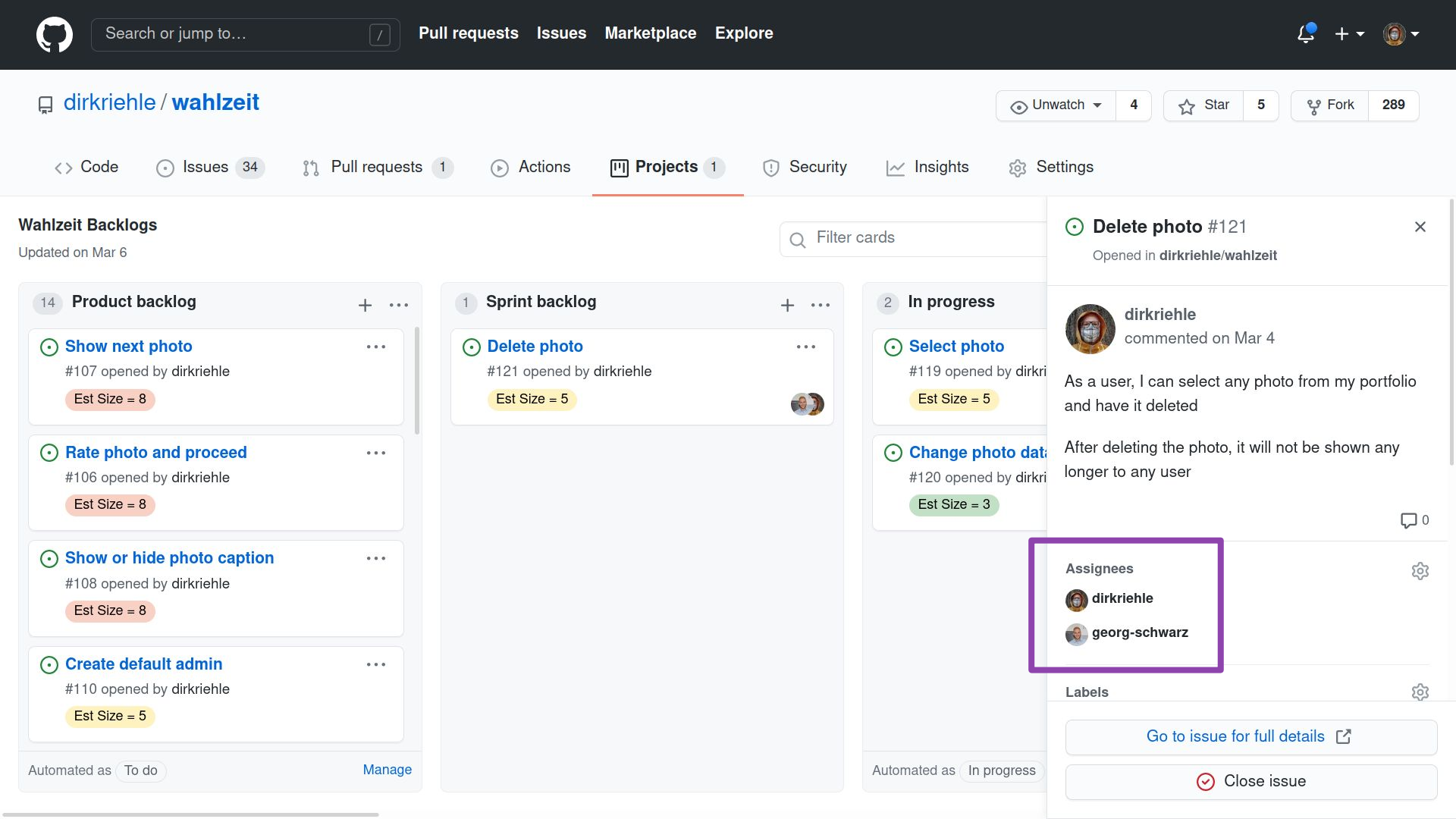
- Added after week 5
- Always the same

Optional additional items

Issue default title:

Assignees:

Labels:


Search or jump to…
Pull requests
Issues
Marketplace
Explore
dirkriehle / wahlzeit
Unwatch 4
Star 5
Fork 289
Code
Issues 34
Pull requests 1
Actions
Projects 1
Security
Insights
Settings
Wahlzeit Backlogs
Updated on Mar 6
Filter cards
14 Product backlog
Show next photo
#107 opened by dirkriehle
Est Size = 8
Rate photo and proceed
#106 opened by dirkriehle
Est Size = 8
Show or hide photo caption
#108 opened by dirkriehle
Est Size = 8
Create default admin
#110 opened by dirkriehle
Est Size = 5
Automated as To do
Manage
1 Sprint backlog
Delete photo
#121 opened by dirkriehle
Est Size = 5
2 In progress
Select photo
#119 opened by dirkri
Est Size = 5
Change photo data
#120 opened by dirkri
Est Size = 3
Automated as In progress
Delete photo #121
Opened in dirkriehle/wahlzeit
dirkriehle
commented on Mar 4
As a user, I can select any photo from my portfolio and have it deleted
After deleting the photo, it will not be shown any longer to any user
0
Assignees
dirkriehle
georg-schwarz
Labels
Go to issue for full details
Close issue

Finishing Features / Closing GitHub Issues

A feature board / GitHub project is just a structured view on GitHub issues

(We'll learn that) a product owner decides on whether a feature is done

- After the decision, everyone can close the issue
- And you should close issues!

You can connect your pull requests to issues and close them automatically

Regular Deliverable: Feature Board

Please initialize your feature board and keep it up-to-date

- For the initial content, meet with your industry partner asap

Commit a screenshot of your feature board

6. Code Repository

Git User Setup

Please use only one account and **one email address and stick to it**

Please configure your name and email address for your local repository

Please make sure this is the same information as on github.com

```
dirk@host$ git config --local user.name "Dirk Riehle"
dirk@host$ git config --local user.email "dirk@riehle.org"
```

Git Commit Sign-off

Please sign-off on your commits as your work using –signoff

This will add your name and email address to the commit message

This signifies that you agree with the project license

```
dirk@host$ git commit -m "Fixed issue #123" --signoff
```

Pair Programming / Co-Authoring

If you are pair programming, please make sure you document this in your commits

- Add “Co-authored-by:” to commit message using **the correct email address**
- Double-check the syntax (otherwise co-authorship will not be recognized)

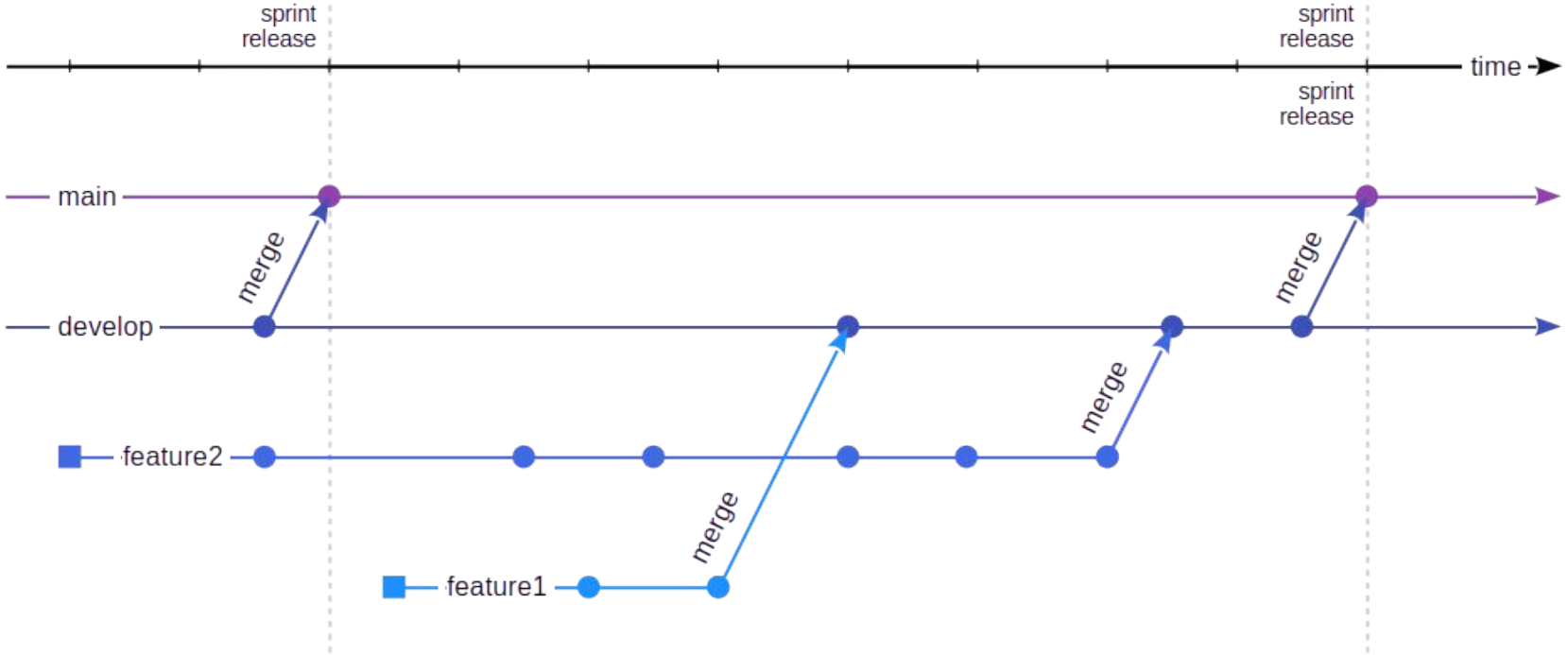
```
dirk@host$ git commit -a -m "Fixed problem
>
>
Co-authored-by: Stefan Buchner <stefan.buchner@fau.de>"
```

Each co-authored-by needs to be on its own line to be recognized

Declaring collaboration in the feature board is not enough

Please find more detailed information on GitHub

Version Control and Feature Branches



Making Your Work Count

We use git, not GitHub [1], to look at your work

If you want your programming work to count,

- Do not squash your commits
- Do not delete branches with relevant work

[1] The GitHub interface / GitHub insights does not display your data correctly

Collaboration Models

If you choose a “review required” collaboration model,

- We will take reviewing as a valid contribution

We suggest that for each merge (pull) request you perform,

- You owe your team one merge request review [1]

[1] Consider adding this to the team contract

Use of Code AIs

The use of code AIs is not only allowed, it is encouraged

You, the human, remain responsible for your work results, though

Under peer review, author and committer share the responsibility

During grading, we do look at the quality of your source code

Sign-offs by code AIs are not accepted and a big red flag

The AMOS Project Licenses

For source code, we use the MIT license

- See https://opensource.org/licenses/MIT

For other data, we use the CC BY 4.0 license

- See https://creativecommons.org/licenses/by/4.0/

License and Copyright Declaration in Files

Please use the REUSE SOFTWARE [1] format to declare license and copyright

```
// SPDX-License-Identifier: MIT
// SPDX-FileCopyrightText: 2010-2021 Dirk Riehle <dirk@riehle.org>
// SPDX-FileCopyrightText: 2019 Georg Schwarz <georg.schwarz@fau.de>
```

Alternatively, just track license and contributors in MIT license file

[1] See https://reuse.software/

Open Source Governance

Do not add copyleft-licensed libraries to your project

Follow these rules of thumb on license choice

- OK: Permissive licenses (MIT, BSD, Apache)
- May be OK: Weakly protective (a.k.a. “weak copyleft”)
- Usually not OK: Strongly protective (a.k.a. “reciprocal” or “copyleft”)
- Never OK: Non-software licenses, no license

Professionals (i.e. companies) use code analyzers to check

7. Imp-Squared Backlog

Impediments and Improvements (Imp-Squared) Backlog

The imp-squared backlog is a slotting system used to

- Manage the processing state of project impediments and improvements

Impediments are non-technical problems that are

- Holding the team and project back

Improvements are non-technical desires to

- Improve team performance

amosproj / **amos202Xss0Y-projX-template** Public template

Pin | Unwatch 2 | Fork 0 | Starred 2

Code | Issues | Pull requests | Actions | **Projects 2** | Security | Insights | Settings

Impediments Backlog

Updated 8 minutes ago

Filter cards | Add cards | Fullscreen | Menu

2 To do

Cards

Cards can be added to your board to track the progress of issues and pull requests. You can also add note cards, like this one!

Added by dirkriehle

Automation

Automatically move your cards to the right place based on the status and activity of your issues and pull requests.

Added by dirkriehle

0 In progress

0 Done

+ Add column

Regular Deliverable: Imp-Squared Backlog

Please initialize your imp-squared backlog and keep it up-to-date

8. Stand-up Emails

Stand-up Emails

Stand-up emails are a communication mechanism that serves to

- Ensure regular updates about each other's work state / progress

When writing a stand-up email, please consider these three topics

1. What did you get done since you last sent a stand-up email?
2. What are your next steps / plans of work to do? [1]
3. What challenges are you facing?

Separate from the stand-up emails, feel free to

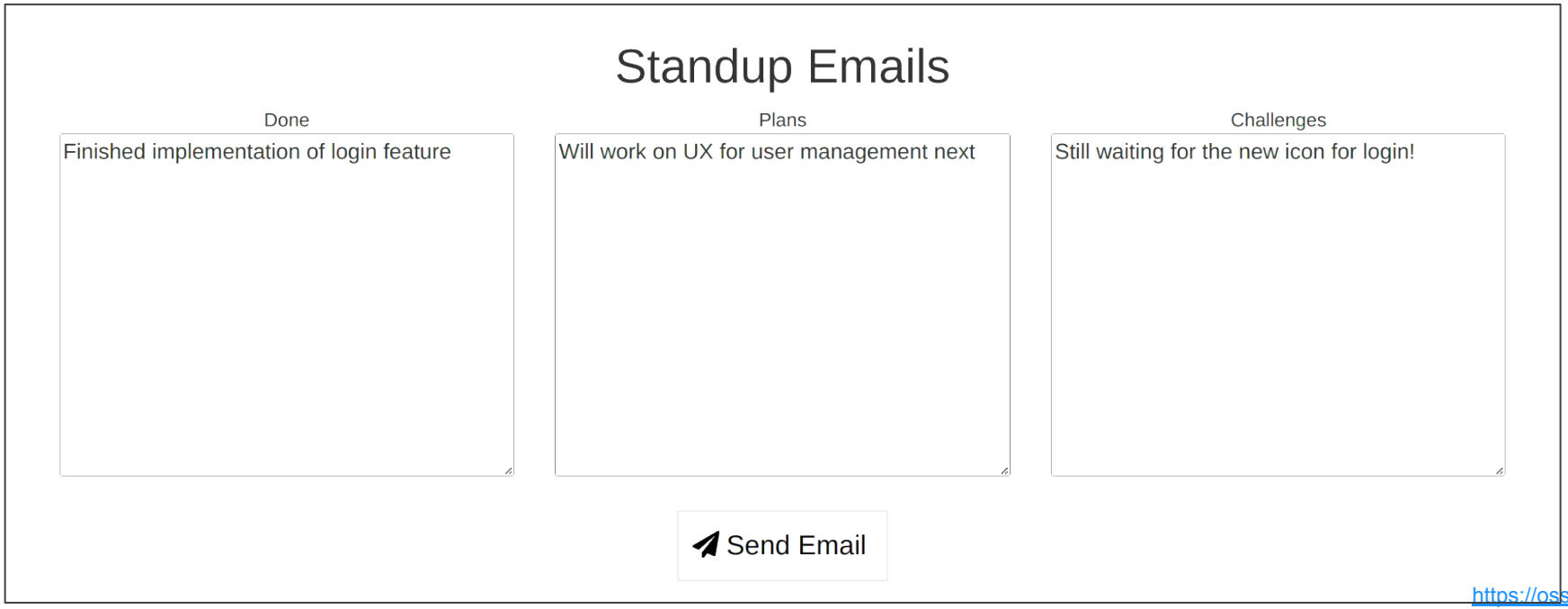
- Have your own communication channels

[1] Don't state the obvious like "do work" or "attend meeting"

Regular Deliverable: Stand-up Emails

Please send stand-up emails using the tool https://happy.uni1.de

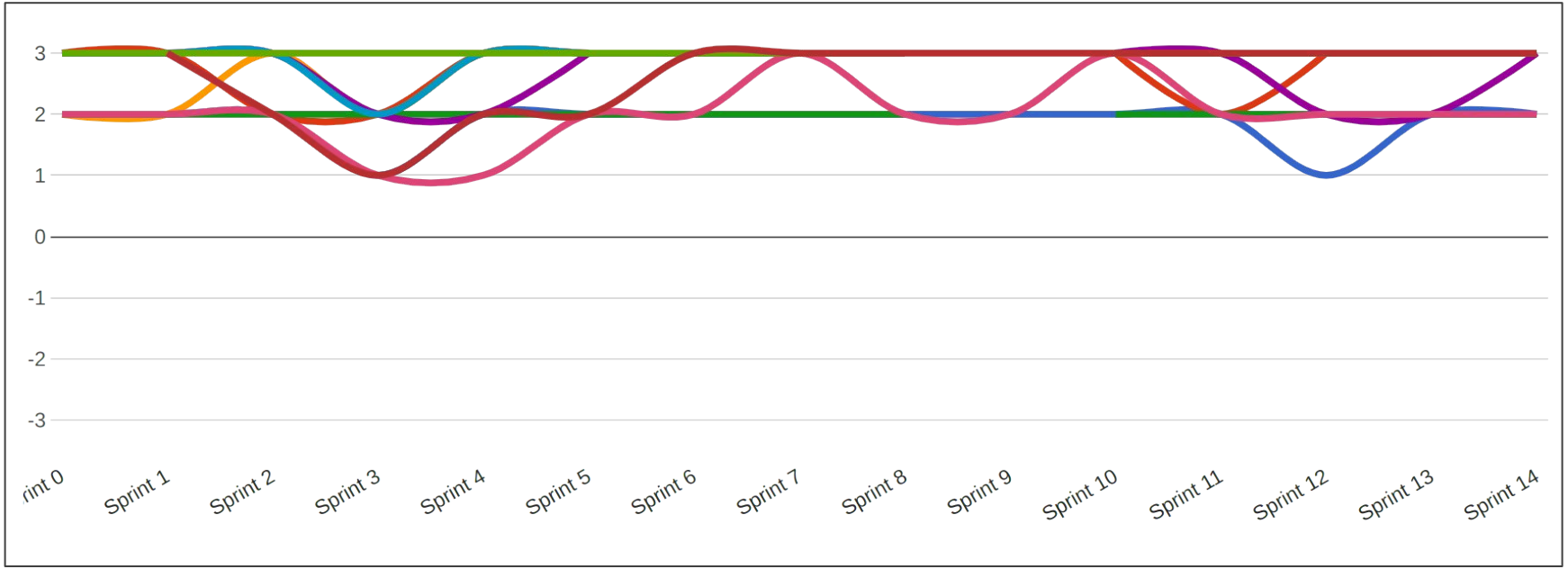
- At least twice in total, on different days of the week



9. Happiness Index

Happiness Index [1]

The happiness index tracks general satisfaction to indicate potential problems



[1] Originally: Emotions Seismograph

Regular Deliverable: Happiness Index

Please indicate your happiness using the tool https://happy.uni1.de

- Every team meeting, including the first and last one, until end of day

Your contributions (your happiness) remains anonymous

Summary

1. Team formation
   a. Student roles
   b. Team contract
   c. Team logo and T-shirt
2. Project tools
   a. Planning document
   b. Feature board
   c. Code repository
3. Team coordination
   a. Imp-squared backlog
   b. Stand-up emails
   c. Happiness index

Thank you! Any questions?

dirk.riehle@fau.de – https://oss.cs.fau.de

dirk@riehle.org – https://dirkriehle.com – @dirkriehle

Legal Notices

License

- Licensed under the CC BY 4.0 International license

Copyright

- © Copyright 2009-2026 Dirk Riehle, some rights reserved